CANDY
DIALOGUE ENGINE

Visual Novels Without Coding

1.0.0

For users who are averse to coding, Candy Dialogue Engine can be used to create and run entire visual novel games through dialogue commands. The process is incredibly simple.

Setting up Candy DE

1. Create a new Godot project.

2. Add Candy DE to the project (as described in the Basic Setup guide).

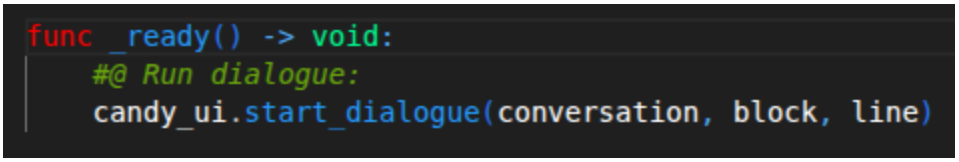
3. In the Godot editor, create a new scene. Make it a simple Node. Call it "Main" and save it as "Main.tscn".

4. Add a new script to that scene. Call it "Main.gd".

5. In Main.gd, delete the _process() function.

6. Under the _ready() function, add this line:

candy_ui.start_dialogue(conversation, block, line)



(Text colors may not match - this is normal)

7. Replace 'conversation', 'block' and 'line' with the Conversation, Block and Line in your dialogue script that the dialogue should start at. For example:

candy_ui.start_dialogue("Conversation_1", "Block_1", 0)

This will make dialogue start immediately when your game launches.

8. Open the Candy DE script named Candy_Engine.gd (found in Candy_DE/Scripts). Expand the "SETTINGS" region if it's collapsed:

```
#°#############################################################################################
#^
#^                                        SETTINGS
#^
#°#############################################################################################
#region ⋯
```

9. Scroll down until you find the 'vn_mode' variable, and set it to 'true':

```
#& VN MODE
#^ VN Mode enable:
#? Visual novel mode. Enables VN commands for displaying actors busts.
#? Does not affect portrait display if true. See 'portrait_mode' to configure portrait display.
var vn_mode = true
```

Later, you can use the rest of the documentation to configure other settings as needed.

10. At this point, the next step is to create your VN in the Candy Dialogue Creator. Once you have done that, export it to a .txt file. Open that file, and copy the entire contents.

Open the Candy_Engine.gd script. Expand the "DIALOGUES" region if it's collapsed. Paste your dialogue script inside the 'running_dialogue' variables, after '=' and replacing the curly brackets:

```
#°#############################################################################################
#^
#^                                        DIALOGUES
#^
#°#############################################################################################
#region

#& Loaded Dialogues
#^ Scripted Dialogues:
var running_dialogue = {}

#endregion
```

Creating your VN

Launch the Candy Dialogue Creator, and follow the Basic Setup guide to create a profile and configure it, if you haven't done so already.

Start Screens

If you're making a full game through Candy Dialogue Engine, you might want to play splash screens, such as your brand logo, and the Candy Arts accreditation. Simply use Video or Image commands for that. See the Media Control guide for more information.

Making Menus

You'll also want a main menu, with a background that expands across the entire screen. For the background, use Background commands. Refer to the Backgrounds guide for more information.

For buttons, such as "New Game", "Options", etc: use a §Choice_List command. See the Player Choices guide for more information.

We won't explain here how to use or configure the commands, this is something you must learn. But here are a few pointers:

- Create a choice category named 'Main' and set it as the menu's main category. Add choices such as "New Game" and "Settings" or "Options" in that category.
- For "Settings" or "Options", create a category, and configure the corresponding choices in the Main category to navigate there. To clarify: The Category "Main" should have a choice named "Settings" which navigates to the category "Settings" when selected.
- In the "Settings" (or "Options") category, add choices for each setting you want players to be able to configure. For example, you can add a choice that increases text writing speed, and a choice that decreases it.
- For such choices, under "Selected", configure a "Continue" transition to another Block. In that Block, use the §Set command to modify the corresponding variables. For example, the "Write Speed Up" choice would transition to a Block that contains a §Set command that increases writing speed by a set amount. Once again, see the rest of the documentation for more details.
- Back in the Main category: for the choice "New Game", under "Selected", set up a "Jump" or "Bridge" transition to another Block. That Block will be where the game will start. You might want to begin with a spoken line (VNs usually begin with someone talking) and you can also add various commands for visual or audio effects, modifying game variables such as character stats, etc.
- Of course, since this is a Visual Novel, you'll also add a lot of VN commands to control busts, and Background commands to display, change or animate backgrounds.

- The Choice_List command can be used to display player choices or for menus, Transition commands are used to change to other Conversations/Blocks...

Saving & Loading

To save and load, you'll want to use the §Export and §Import commands (see the Export-Import guide for more information).

Listing the variables that the §Export command should export will be your responsibility. Since this is your game, you'll be creating those variables yourself and you should know which ones need to be saved.

If you want to save the current conversation, block and line, you can export the variables named 'conversation_tracker', 'block_tracker' and 'line_tracker', found in the corresponding instance of the Candy_Engine.gd script: these are automatically updated each line. Note, however, that this will save the Conversation/Block/Line of the §Export command.

For loading, you should create a special dialogue Block that has all the necessary commands to reload game progress, and which ends with a transition to the dialogue line where things should resume.

When loading (with the §Import command), the variables you exported are updated with the values stored in the export file. However, keep in mind that this doesn't restore all game progress automatically.

For example, if you saved the images assigned to character busts, §Import can restore the images, but won't resume playing their animations (if any were playing when the game was saved). It also won't load the bust scene in the first place.

For situations like that, you should configure the §Import command to import to a dictionary rather than directly to the original variables. You can then use various commands, such as §VN_Scene, to restore whatever needs restoring: the bust scene, backgrounds, media...

Of course, loading is a lot easier if you carefully choose when players can save: e.g. when animations and media are not playing. Even better if the §Export commands is located at the start of a Block, accessed with a §Jump transition, and the §Import command immediately follows it, hence saving and loading before any other lines in the Block are processed.

You'll have show a bit of creativity in order to implement a proper save/load system that works entirely through dialogues. It's not necessarily difficult, but it requires being comfortable with the various commands that Candy DE offers, and not being afraid of doing some trial-and-error.

Once you have §Export and other commands setup to save properly, remember that you can create a preset for it in the Candy Dialogue Creator. This way, you won't have to reconfigure the same commands over and over each time you want to let the player save.

We might make some tutorials about setting up a save/load system in the future, but it will be after Candy DE is out of pre-release.